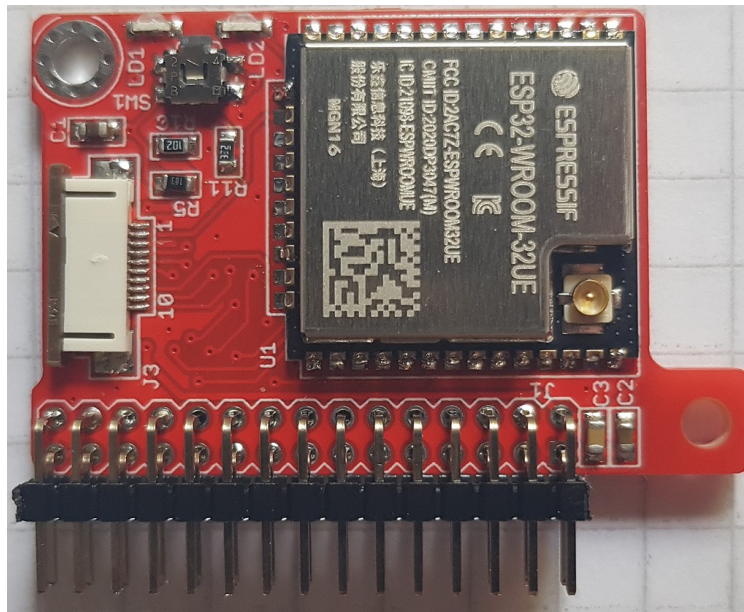


ESP32 Miniboard

ESP32 processor for embedded applications



Description

the product

The product consists of a redesigned format of the usual ESP32 devkit, allowing the usage in custom application mainboards in vertical or horizontal position, and in as small as possible enclosures like DIN rail modules, waterproof cases, wall mounting boxes.

To achieve the dual position feature, a single double row connector will be provided, either straight for horizontal mount, or right-angle for vertical.

Its size, 26x40mm, has also been reduced (smaller than a pocket matchbox)

Github repository

<https://github.com/JLB-ZH/ESP32-Slimboard-26x40>

The board

The board is fitted with an ESP32-WROOM-32UE (the E model can be used if no external antenna is needed) with 16MB of flash memory.

There is no USB interface nor 3.3V regulator on-board, since that feature is only used a very few times in the product life (an external adapter need to be used for programming the processor at first time, OTA can be used further)

It features a 2x15 pin male connector with a 2.0mm pitch, allowing its use on prototyping or production platform.

Two sideleds are mounted on the PCB.

Specifications :

- dimensions : 26x40x7mm
- Soc ESP32-WROOM32-E (or UE)
- external 3.3V power supply
- 1x 2x15x2.0mm male connector (Straight for horizontal or right-angle for vertical mounting)
- Reset button
- 2x LED indicators
 - red LED on GPIO0
 - green LED on GPIO2
- 1x 10x0.5mm ZIF connector for Uart or Jtag programming and I²C/SPI expansions
- Communication
 - Wifi
 - Bluetooth
 - I²C
 - Can
 - Uart
 - 1 Wire

Adapter boards are available for use on prototyping boards.

Connectivity

2x15 connector

Connector pin assignment

The *description* columns correspond to an example of application : the F1ATB photovoltaic router

description	App signal	ESP32 gpio	pin	pin	ESP32 gpio	App signal	description
0V power supply	GND		1	2		+3,3V	Power supply +3.3V
Input 1, dig ana or dif	in1	36	3	4	39	in2	Input 2, dig, ana or dif
Input 3, dig or ana	in3	34	5	6	35	in4	Input 4, num or ana
PWM output	PWM	22	7	8	23	ZC	Zero crossing or pulse input
Uart 0	RX0	3	9	10	26	RX1	Uart 1, USB
	TX0	1	11	12	27	TX1	
Ouput 1	out1	18	13	14	19	out2	Output 2
Output 3	out3	32	15	16	33	out4	Output 4
I ² C	SDA	14	17	18	21	1wire	1wire bus
	SCL	13	19	20	25	out5	Output 5
Output 6	out6	17	21	22	12	out7	Output 7, led3 or RS485 TXEN
Bus CAN	CAN_RX	4	23	24	15	Led1	Output led1
	CAN_TX	5	25	26	16	Led2	Output led2
Boot (gpio0) or D-	BOOT	0	27	28		EN	EN (reset) or usb D+
0V USB	Gnd1		29	30		VINUSB	USB +5V

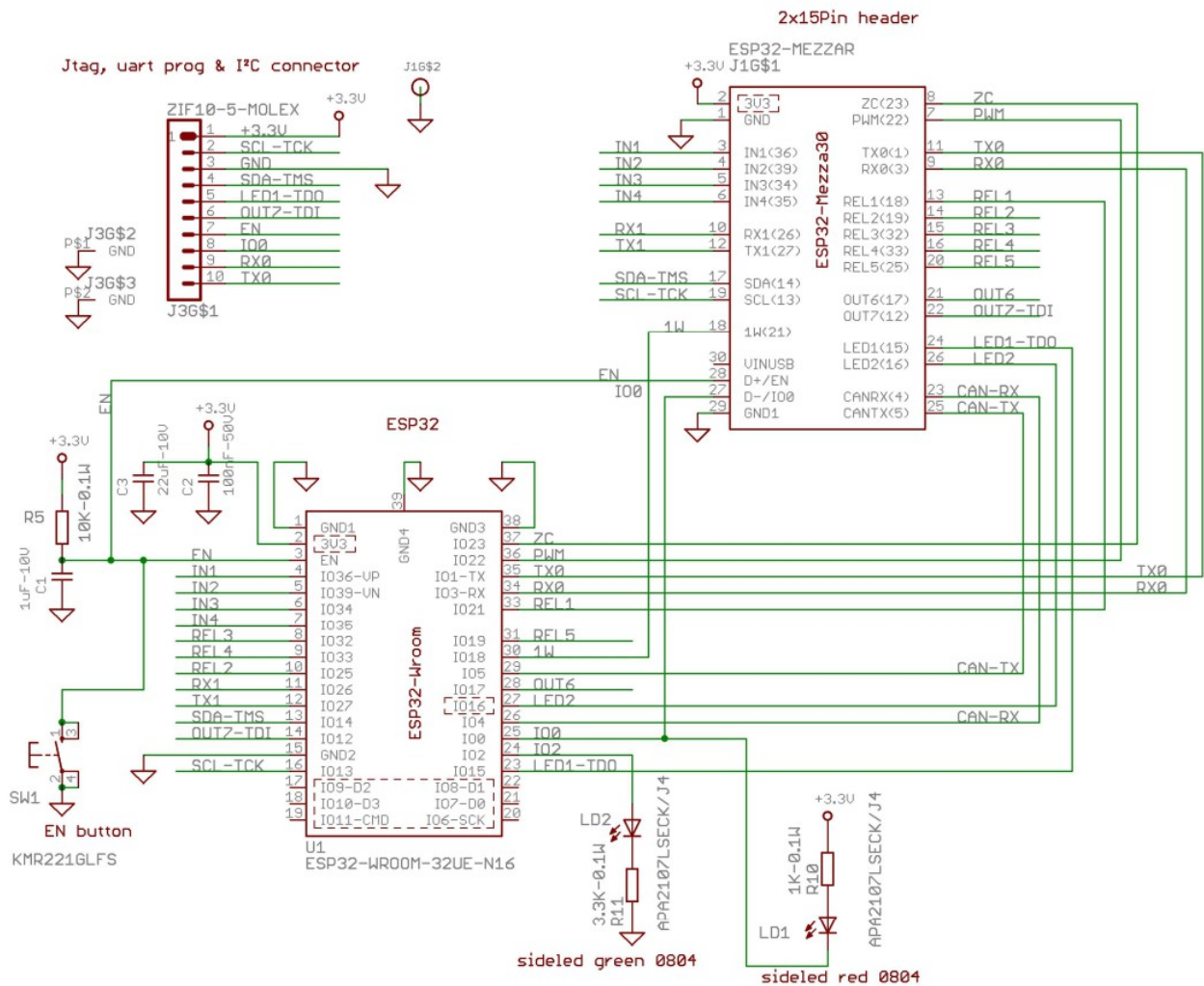
Zif connector

The ZIF connector allows :

- programming the ESP32 by Jtag with the ESPPROG2 board + additional adapter board
- programming the ESP32 by Uart autoprog mode with the ESPPROG2 board + additional adapter board
- connecting any I²C péripheral custom board or display directly to the ESP32 board
- Adding an external Sdcard or SPI memory

Pin	GPIO	Fonction
1	VDD	+3,3V
2	GPIO13	I ² C CLK / JTAG TCK / HSPI D
3	GND	GND
4	GPIO14	I ² C DATA / JTAG TMS / HSPI CLK
5	GPIO15	JTAG TDO / HSPI CS0
6	GPIO12	JTAG TDI / HSPI Q
7	EN	Reset / Enable
8	GPIO0	Boot mode (IO0)
9	GPIO3	UART0 RX
10	GPIO1	UART0 TX

Schematic



Bom

Measures in mm, 2 holes Ø 2.6mm.



The vertical mount footprint differs from the horizontal one in the position of pin 1. Caution should be taken when designing the baseboard, and using a vertical card on an horizontal socket or vice versa will destroy the ESP32 module.

Horizontal mounting



page 6

Software

Onboard Leds

Note :

the on-board **green** LED is connected to GPIO2, and is activated with a HIGH level

the on-board **red** LED is connected to GPIO0, and is activated with a LOW level

When using Arduino IDE, GPIO2 is considered as the ESP32 devkit onboard LED, and is driven by the libraries to reflect the Wifi Status.

In order to disable this function and take control of GPIO2 for your application usage, or switch off this LED, it is necessary to add this piece of code at the beginning of the setup() function :

```
#define Red_led 0 // led 1 on ESP32 wroom board
#define Green_led 2 // led 2 on ESP32 wroom board

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  // reinit gpio2 pin
  //.....
  pinMode(Green_led, OUTPUT);
  GreenLedOFF();
}
```

Various Applications

OpenPLC

An opensource PLC, supported through the following website

<https://autonomylogic.com/>

It is possible to add custom functions into the PLC program.

It can easily be interfaced with HomeAssistant through the Modbus integration

F1ATB Photovoltaic router

[Solar Energy Router to manage overproduction \(DIY\) – F1ATB](#)

A system that can use the available power from solar panel to drive any resistive load, leaving priority to the other installation equipments.

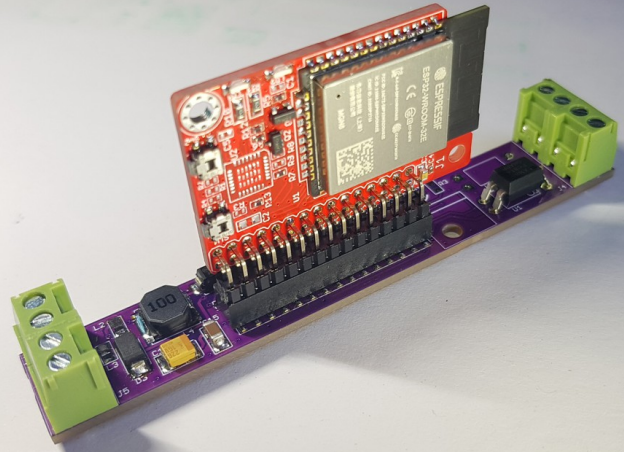
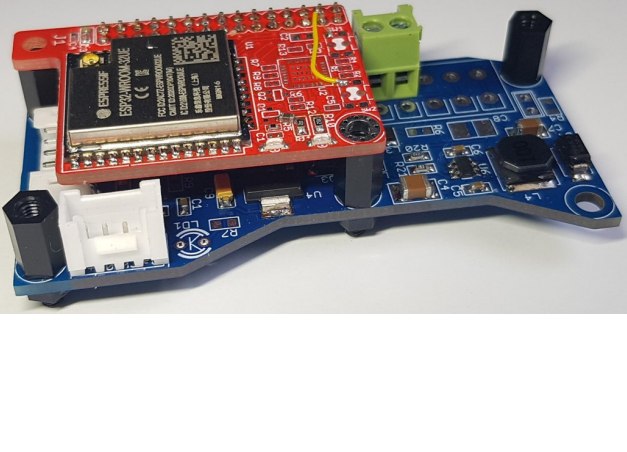
CAN bus gateway

Another interesting project is the HAPCAN one, for lighting management over CANbus, where any device can interact with any of the other devices connected to the bus, even itself.

The ESP32 board can be used as a CAN gateway, or as a button interface, if installed inside a wall mounting box

Enclosures

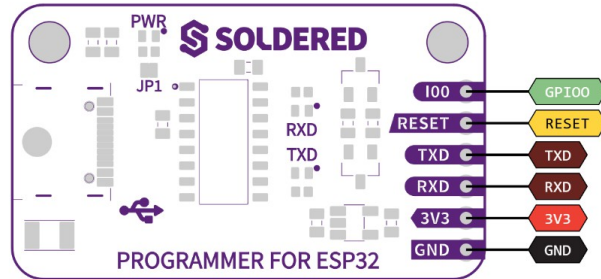
The board can be installed on a custom baseboard, either in vertical position, for instance in slim Din rail enclosures, or even horizontally position if space enough.

One module Din rail baseboard	Custom sensor baseboard
	
	

Programming

Software can be programmed into the ESP32 card by several ways.

SOLDERED Connect card



this small and low cost card with USB-C type connector uses a CH340C chip, and generates the IO0 and EN signals necessary to reset and boot the ESP32.

A simple 1x6 pin header must be provided on the base board, and a 5 or 6 wires cable provided between the programming card and the baseboard

pin	Prog card	Esp32 card	2x15 connector	ESP32 GPIO	description
1	Gnd	Gnd	1		0V power ground
2	+3.3V	+3.3V	2		Do not connect if 3.3V to ESP32 is provided by the baseboard
3	RXD	TX0	11	GPIO1	RXD of CH340, TX0 of ESP32
4	TXD	RX0	9	GPIO3	TXD of CH340, RX0 of ESP32
5	RESET	EN	28		EN
6	GPIO0	IO0	27	GPIO0	IO0

ESPprog2 card

an Espressif esp32 based programming card with a Jtag interface and an Uart programming interface.

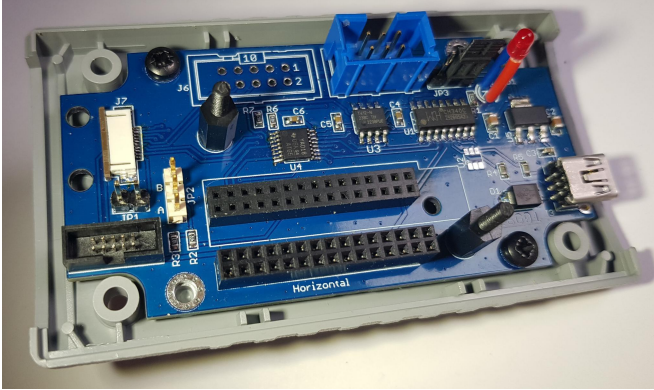


Zif Adapter card

this multifunction custom card provides the Zif connector and can program the ESP32 card in either vertical or horizontal position, and be interfaced with the ESPPROG2 card or SOLDERED Connect card.

It also can be directly connected to an USB port for programming (onboard CH340-C)

An option jumper can be installed to provide +3.3V power to the Zif connector.

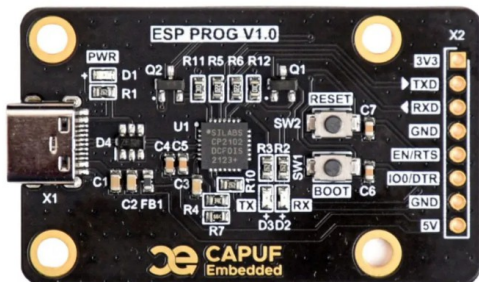


CAPUF card

ESP32 Programmer V1: ESP32 Programmer

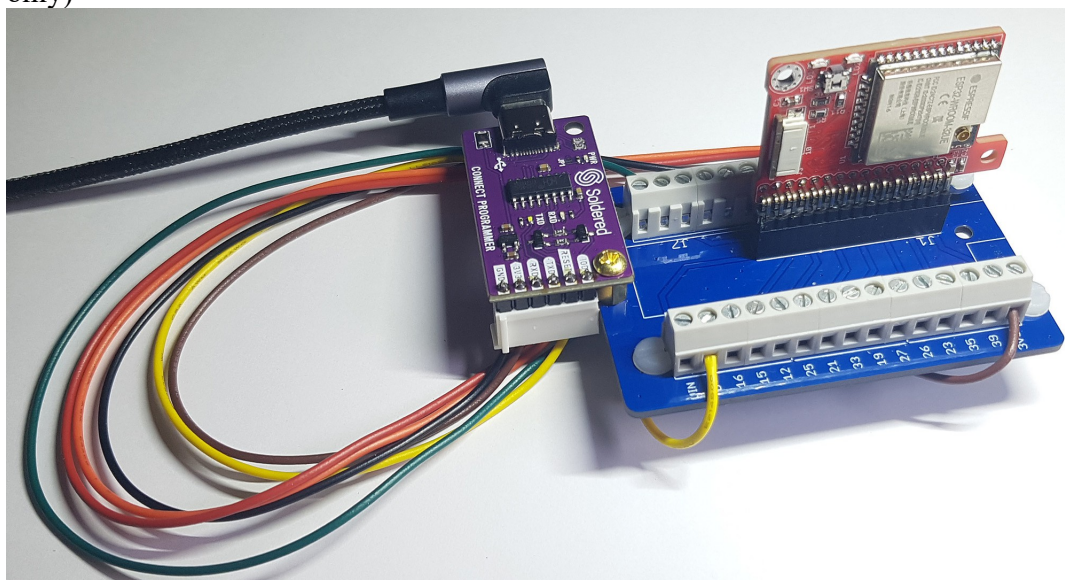
a 5 or 6 wires custom cable should be made as described above.

This card uses a CP2102 chip for Usb to Uart conversion.

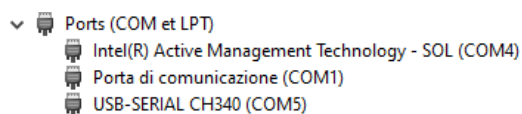


Custom baseboard with connection blocks

the following custom programming platform uses the SOLDERED 'Connect' card (vertical mode only)



once connected to the computer on USB port, the control panel should show an additional COM port device : COM5



description	App signal	ESP32 gpio	pin	pin	ESP32 gpio	App signal	description
0V power supply	GND		1	2		+3,3V	Power supply +3.3V
			3	4			
			5	6			
			7	8			
Uart 0	RX0	3	9	10			
	TX0	1	11	12			
			13	14			
			15	16			
			17	18			
			19	20			
			21	22			
			23	24			
			25	26			
Boot (gpio0)	BOOT	0	27	28		EN	EN (reset)
			29	30			

a simple software example

```
// =====
// a simple example that makes the 2 onboard leds blink asynchronously
// =====

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// on-board LEDs
#define LED_red 0 // GPIO0, active low
#define LED_green 2 // GPIO2, active high

// red LED blinking task
//-----
void BlinkRedLedTask(void *pvParameters) {
const TickType_t xDelay = pdMS_TO_TICKS(500); // 500ms delay
while (1) {
    ledON(LED_red);
    vTaskDelay(xDelay); // Delay for 500ms
    ledOFF(LED_red);
    vTaskDelay(xDelay); // Delay for 500ms
}
}

// initialisation
//-----
void setup() {
    // put your setup code here, to run once:
    pinMode(LED_green, OUTPUT); // this also disable the GPIO2 from beeing the Wifi status led
    ledOFF(LED_green);

    pinMode(LED_red, OUTPUT);
    ledOFF(LED_red);

    Serial.begin(115200);
    Serial.println("Start blinky task");
    xTaskCreate(BlinkRedLedTask, "blink_led_red", 2048, NULL, 1, NULL);
}

// main loop
//-----
void loop() {
    ledON(LED_green);
    delay(100);
    ledOFF(LED_green);
    delay(900);
}

// functions
//-----
```

```
void ledON(int led) {  
  if (led == LED_green)  
  {  
    Serial.println("LED_green ON");  
    digitalWrite(led, HIGH);  
  }  
  else  
  if (led == LED_red)  
  {  
    digitalWrite(led, LOW);  
  }  
}
```

```
void ledOFF(int led) {  
  if (led == LED_green)  
  {  
    Serial.println("LED_green OFF");  
    digitalWrite(led, LOW);  
  }  
  else  
  if (led == LED_red)  
  {  
    digitalWrite(led, HIGH);  
  }  
}
```